

JO YOUNG DAE

Cloud Infrastructure Engineer / Platform Engineer

운영을 고려한 설계로 안정적인 플랫폼을 만드는 엔지니어

AWS

Terraform

Kubernetes

Platform

GitOps

Private Cloud

운영을 먼저 생각하는 엔지니어

문제를 임시로 해결하는 것이 아니라 원인을 끝까지 분석하여 재발하지 않는 구조를 만드는 것을 중요하게 생각합니다. Terraform 기반 AWS 인프라 구축 프로젝트와 KVM+libvirt 기반 On-Premise Kubernetes Platform 프로젝트를 수행하며 운영 중심 설계와 플랫폼 자동화 경험을 쌓았습니다.

Operation First
구축 이후의 운영과
장애 대응을 먼저
고려합니다.

Problem Solving
문제를 임시로 넘기지
않고 원인을 끝까지
추적합니다.

GitOps
반복 작업을 줄이고
운영 효율을 높입니다.

Collaboration
운영 시나리오와
근거를 바탕으로 팀과
의사결정합니다.

Technical Skills

Cloud

AWS Azure

IaC

Terraform

Container

Docker Kubernetes

Virtualization

KVM libvirt containerd Cilium

AI

Azure OpenAI

Programming

Python Node.js TypeScript

Version Control

Git GitHub

KT Cloud Infra Bootcamp

2025.12 ~ 2026.07

클라우드 인프라 전문 과정에서 AWS, Terraform, Docker, Kubernetes, CI/CD, Private Cloud, Azure 기반 실습과 프로젝트를 수행했습니다.

AWS

Terraform

Docker

Kubernetes

CI/CD

Private Cloud

Azure

Projects

PROJECT 01

Terraform 기반 AWS Infrastructure

Cloud Infrastructure Engineer

2026.01.26 ~ 2026.03.05

Contribution 50%

PROJECT 02

Kubernetes Internal Developer Platform

AIOps Engineer

2026.03.10 ~ 2026.04.10

Contribution 60%

Terraform 기반 AWS Infrastructure



Demo QR

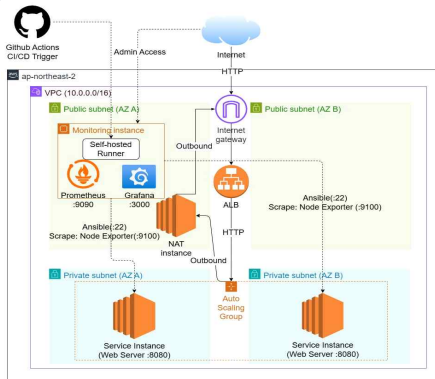
Overview

- Terraform 기반으로 AWS 인프라를 코드화하여 반복 가능한 구축 환경을 구현했습니다.
- 운영 환경을 고려한 네트워크 설계와 Auto Scaling 구성을 통해 안정적인 인프라 구축을 목표로 했습니다.

Background

- 운영 환경에서는 동일한 인프라를 반복적으로 구축해야 하는 상황이 자주 발생합니다.
- 수작업 구성은 환경 차이와 설정 오류로 이어질 수 있어 Infrastructure as Code를 적용했습니다.

AWS Architecture



Infrastructure Components

Terraform

ALB

ASG

EC2

NATInstance

Ubuntu

Nginx

Git

PR

기여 요약

- Terraform 기반 AWS 인프라 구축 VPC,
- EC2, ALB, Auto Scaling 구성
- 운영 안정성을 고려한 NAT Instance 역할 분리 제안

왜 이렇게 설계했는가

Terraform

Infrastructure as Code 적용으로 반복 가능한
인프라 구성

ALB

가용성을 고려한 트래픽 분산

ASG

확장 가능한 서비스 인스턴스 구성

NAT Instance

운영성과 장애 분석을 고려한 역할 분리

TROUBLESHOOTING

운영 안정성을 위한 역할 분리

Problem

비용 절감을 위해 NAT Instance와 Private Cloud Instance를 통합하자는 의견이 있었습니다.

Risk

장애 발생 시 역할이 혼재되면 원인 분석과 복구가 어려워질 수 있다고 판단했습니다.

Solution

운영 시나리오를 기준으로 역할 분리의 필요성을 설명하고 팀 논의들 통해 구조를 수정했습니다.

Result

운영 안정성과 장애 대응성을 고려한 인프라 구조를 설계할 수 있었습니다.

LESSONS LEARNED

laC는 자동화보다 운영 표준화에 가깝다

Terraform을 통해 인프라를 코드로 관리하면서, **laC**는 단순히 리소스를 빠르게 만드는 기술이 아니라 동일한 운영 환경을 반복 가능하게 만드는 과정이라는 점을 배웠습니다.

Kubernetes Internal Developer Platform

Overview

- 실제 기업 환경을 가정해 개발자에게 편리한 개발 플랫폼을 제공하고, 운영자에게는 반복적인 운영 업무를 줄일 수 있는 IDP를 구축했습니다.
- Discord ChatOps, Backstage AI Assistant, Azure OpenAI를 활용해 Kubernetes 운영 정보를 자연어로 조회할 수 있도록 설계했습니다.

Background

- Kubernetes와 MSA 환경이 확대되며 개발자는 여러 운영 도구를 직접 사용해야 했고, 운영자는 반복적인 문의와 장애 대응 업무를 수행해야 했습니다.
- 이러한 운영 부담을 줄이기 위해 자연어 기반 운영 지원 플랫폼을 구현했습니다.

KVM 기반 Virtual Machine 4대 구성

단일 고사양 서버를 KVM + libvirt 기반으로 가상화하여 Control Plane 1대와 Worker Node 3대로 분리하고, Kubernetes Cluster 구축을 위한 On-Premise 환경을 구성했습니다.

Virtualization Structure

Host Server

Ubuntu 24.04 · 32 Core · 128GB RAM

KVM + libvirt

Linux Native Hypervisor

k8s-cp · Control Plane

k8s-w1 · Worker 1

k8s-w2 · Worker 2

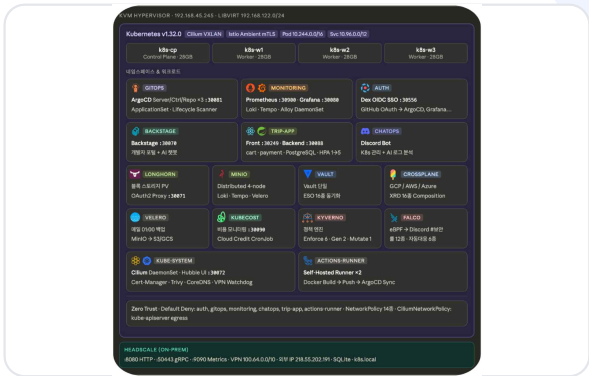
k8s-w3 · Worker 3

VM Resource Planning

VM	Role	CPU	RAM	Disk
k8s-cp	Control Plane	4	16GB	100GB
k8s-w1	Worker 1	8	32GB	400GB
k8s-w2	Worker 2	8	32GB	400GB
k8s-w3	Worker 3	8	32GB	400GB

On-Premise Platform

PROJECT 02 ARCHITECTURE



CORE FEATURE

Discord ChatOps



조영대 2026-03-18 오후 5:23

파드목록 보여줘



@조영대 파드목록 보여줘



딸깍봇 2026-03-18 오후 5:24

현재 전체 Pod는 110개입니다.

- auth/dex-8cc65dd9b-gxdq8 (Running)
- backstage/backstage-backend-6597bb5c6c-wzcjw (Running)
- backstage/backstage-postgresql-698bdfd4f9-5mr44 (Running)
- cert-manager/cert-manager-6555fc48f6-stcvf (Running)
- cert-manager/cert-manager-cainjector-5686986979-mddzw (Running)
- cert-manager/cert-manager-webhook-656bdfd76f-8nb9p (Running)
- crossplane-system/crossplane-6666cdd67d-l7m7j (Running)
- crossplane-system/crossplane-rbac-manager-8d7b65fb8-f26xn (Running)
- crossplane-system/provider-gcp-dd5220cb2129-6cbb446b94-crq7l (Running)
- default/node-debugger-k8s-cp-7qm9f (Completed)

기능

- Pod 상태 조회
- 로그 조회
- 장애 분석
- Service 조회

CORE FEATURE

Backstage AIAssistant

aws 서비스를 띄워보고 싶은데 잘 몰라
어떻게 할까?

추천 템플릿: aws-service-wizard

이유:

AWS 환경에서 서비스를 처음 생성하는 경우, aws-service-wizard 템플릿이 단계별로 필요한 설정을 안내해 주어 쉽게 서비스를 만들 수 있습니다. 인프라 구성과 서비스 배포를 함께 도와줍니다.

생성 방법:

1. Backstage에서 aws-service-wizard 템플릿을 선택합니다.
2. 서비스 이름, AWS 리전, 필요한 리소스 등을 입력합니다.
3. 템플릿 안내에 따라 설정을 완료하고 서비스를 배포합니다. 완료 후 AWS 콘솔이나 Backstage에서 상태를 확인할 수 있습니다.

역할

- 포털 기반 자연어 질의
- Kubernetes 운영 지원
- 서비스 생성 템플릿 추천
- 개발자 셀프서비스 경험 제공

ArgoCD Pipeline



Technical Decisions

- Discord: 운영자가 익숙한 협업 도구를 활용해 접근성 향상
- Backstage: Internal Developer Platform 역할 수행
- Azure OpenAI: 자연어 기반 운영 질의와 장애 분석 지원
- Kubernetes API: 실시간 클러스터 운영 정보 제공
- Fallback: previous → current → describe 순서로 분석 가능성 확보

운영 중단을 막기 위한 예외 처리 구조

Kubernetes API

API 오류가 Bot 중단으로 이어지는 문제를 try-catch와 fallback 처리로 개선했습니다.

Azure OpenAI

Proxy, API Key, Deployment Name, CORS 문제를 순차적으로 분석하고 해결했습니다.

Natural Language

잘못 분류되는 질문을 intent 우선순위 조정과 토큰화 방식으로 개선했습니다.

Fallback

로그가 없을 경우 previous → current → describe 순서로 분석 가능성을 확보했습니다.

CONTACT

안정적인 플랫폼을
함께 만들어가겠습니다.

GitHub: <https://github.com/YoungDae-Jo>

Email: gsxwdv961@gmail.com

Portfolio: youngdae-cloud-portfolio.vercel.app



Terraform Demo



IDP Demo